

2.4 ppt

2.4 工具治理和安全边界

核心路线：运行参数 → 权限状态机 → Runtime → 四项核心测试 → 验收

一、导论

1.1 直接执行工具的问题？

我们刚把瑞幸开放平台的六个核心API封装成Agent工具，直接暴露给模型调用。Agent工程师的直觉告诉我们，每一行代码都在失控的边缘。

`queryShopList` 查附近门店，模型可能把"南京东路"传成"南京西路"，查不到结果。更麻烦的是，Tool Result只返回了一个空列表，没有任何错误码，模型不知道是"参数错了"还是"真的没有门店"，用同样的参数反复重试，陷入死循环，浪费token还让用户干等。这暴露的问题是：Tool Result缺少稳定错误码，Agent Loop无法做出正确决策。

`createOrder` 确认下单、真金白银扣钱，网络一卡顿，Agent的HTTP客户端超时抛异常，捕获异常后自动重试三次。三次重试都"成功"了——瑞幸收到了三笔订单，用户的账户被冻结了三笔钱。用户以为第一笔就成功了，后面两笔完全不知情，找客服才发现被扣了三倍的钱。这暴露的问题是：读操作和写操作使用了相同的重试策略，非幂等写操作被盲目重放。

`queryOrderDetailInfo` 查订单详情，Token一旦泄露，或者模型在参数里伪造了别人的 `user_id`，说"帮我查一下user_123的订单"，而当前Token对应的用户是user_456。如果系统不做二次校验，B用户就能看到A用户的订单信息，这是严重的隐私事故。这暴露的问题是：身份和授权事实由模型参数传递，而不是由服务端上下文决定。

`cancelOrder` 取消订单或退款，用户说"帮我取消最近一单"。模型查到了最近一单，调了 `cancelOrder` 接口，但瑞幸返回了报错——因为那杯咖啡五分钟前已经被取餐了。模型拿到报错信息也不知道该怎么办，用户更懵："我就想退个款，为什么不行？"这暴露的问题是：业务状态校验缺失，Handler被调用时订单状态已经不允许该操作了。

本质暴露：LLM只负责生成"我想调哪个接口、填什么参数"的候选动作。Agent工程师的责任，是把候选动作放进确定的工程护栏里，再放行。

- **工具治理解决：**当前这一次调用，有没有资格进入真实世界
- 协议连通 ≠ 安全边界建立

四类典型故障：

- 1. 权限逃逸
- 2. 参数篡改
- 3. 幂等缺失
- 4. 绕过规则

本质：模型只产生“候选动作”，系统却错误地将候选动作当成了“执行许可证”

1.2 治理流程：从候选动作到可控执行



1.3 最小可用的治理骨架

- 策略放在哪里（规则定义的位置）
- 可信事实从哪里来（身份、租户、审批状态等上下文）
- 拒绝发生在哪个时机（执行前拦截，而非执行后补救）
- 结果回写到何处（审计日志、状态回滚、Loop 决策）

二、先看演示结果：什么叫“可控执行”？

工具治理的目标，不是让所有调用都成功，而是让应该执行的动作成功，让不该执行的动作在副作用发生之前停下来，并给 Agent Loop 返回一个稳定、可解释、可追踪的结果。

代码块

```
1  {"tool_call_id":"call_01","tool_name":"get_order","ok":true,"action":"allow","code":"OK","content":
  {"status":"paid","refundable":399.0,"customer_email":"***@***","access_token":"***"}}
2  {"tool_call_id":"call_02","tool_name":"create_refund","ok":false,"action":"confirm","code":"APPROVAL_REQUIRED","content":"需要确认本次具体动作"}
3  {"tool_call_id":"call_03","tool_name":"create_refund","ok":true,"action":"allow","code":"OK","content":
  {"refund_id":"ref_9001","idempotency_key":"call_03","tenant_id":"tenant_a","order_id":"ord_1001","amount":399.0,"status":"accepted"}}
4  {"tool_call_id":"call_04","tool_name":"create_refund","ok":false,"action":"deny","code":"INVALID_ARGUMENT","content":[{"path":"user_id","message":"Extra inputs are not permitted"}, {"path":"approved","message":"Extra inputs are not permitted"}]}
5  {"tool_call_id":"call_05","tool_name":"run_shell","ok":false,"action":"deny","code":"DENY_RULE","content":"命中 deny 规则"}
6  {"tool_call_id":"call_06","tool_name":"create_refund","ok":false,"action":"deny","code":"PLAN_MODE_DENIED","content":"plan 模式禁止写操作和 Shell"}
7  {"side_effects":{"refund_executions":1,"shell_executions":0},"audit_records":8}
```

解决方案：

工具治理的目标，不是让所有调用都成功，而是让应该执行的动作成功，让不该执行的动作在副作用发生之前停下来，并给 Agent Loop 返回一个稳定、可解释、可追踪的结果。下面这六条日志，是一条完整的瑞幸点单链路从查询到下单的治理轨迹。

第一条，查询门店成功。Agent调用了 `queryShopList`，用户想查“南京东路”附近的门店。Runtime校验通过，返回了门店信息。但注意，门店的详细地址被脱敏了，只保留了路名和距离，这是为了防止敏感地理位置信息进入模型上下文后被意外泄露。

第二条，下单请求 — 需要用户确认。Agent调用了 `createOrder`，在诺布中心店下一杯橙C美式大杯，金额20元。Runtime判定这是高风险资金操作，返回 `APPROVAL_REQUIRED`，调用暂停，等待用户在界面上点击“确认支付”。模型只发起了申请，真正的钱还没扣。

第三条，用户确认后下单执行成功。用户在前端点击了确认，`approval_id` 随请求重入Runtime。审批校验通过——用户、门店、商品、金额全都对得上，Handler真正调用了瑞幸的下单接口，返回了订单号 `RX20260829001`，状态为待支付。注意，这个调用携带了幂等键 `call_03`，同样的请求再发一次会被拦截。

第四条，伪造身份被参数校验拦截。模型在 `createOrder` 的参数里偷偷塞了 `user_id` 和 `approved` 字段，想冒充其他用户绕过确认。Pydantic Schema配置了 `extra="forbid"`，直接报 `INVALID_ARGUMENT` 拒绝。Handler完全没有执行，瑞幸接口零调用，用户的账户安全。

第五条，已取餐的订单取消被业务规则拦截。用户想取消订单，Agent调用了 `cancelOrder`，但该订单五分钟前已经被取餐了。业务预检查询了订单状态，发现是 `picked_up`，直接返回 `BUSINESS_RULE_DENIED`，Handler没有执行。避免了无效调用和潜在的资损风险。

第六条，Plan模式下写操作被拒绝。当前模式是 `plan`，也就是只允许查询和分析，不允许真实下单或退款。Agent发出了 `createOrder` 请求，Runtime在执行层直接拦截，返回 `PLAN_MODE_DENIED`。这确保在规划阶段的任何“试探性下单”都不会产生真实副作用。

这就是“可控执行”的含义——该成的成，不该成的在执行之前停下来。工程师看这份日志，能清楚地知道每一笔订单的来源和每一步决策的依据。

2.1 六条结果逐条解读

第一条（查询订单成功）

第二条（退款申请 — 需人工确认）

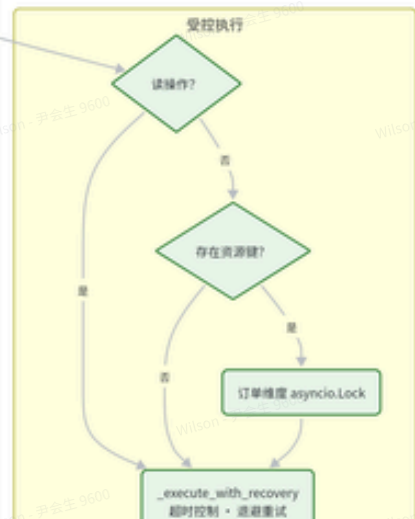
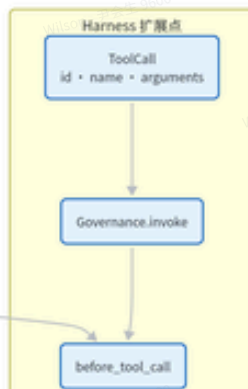
第三条（退款执行成功）

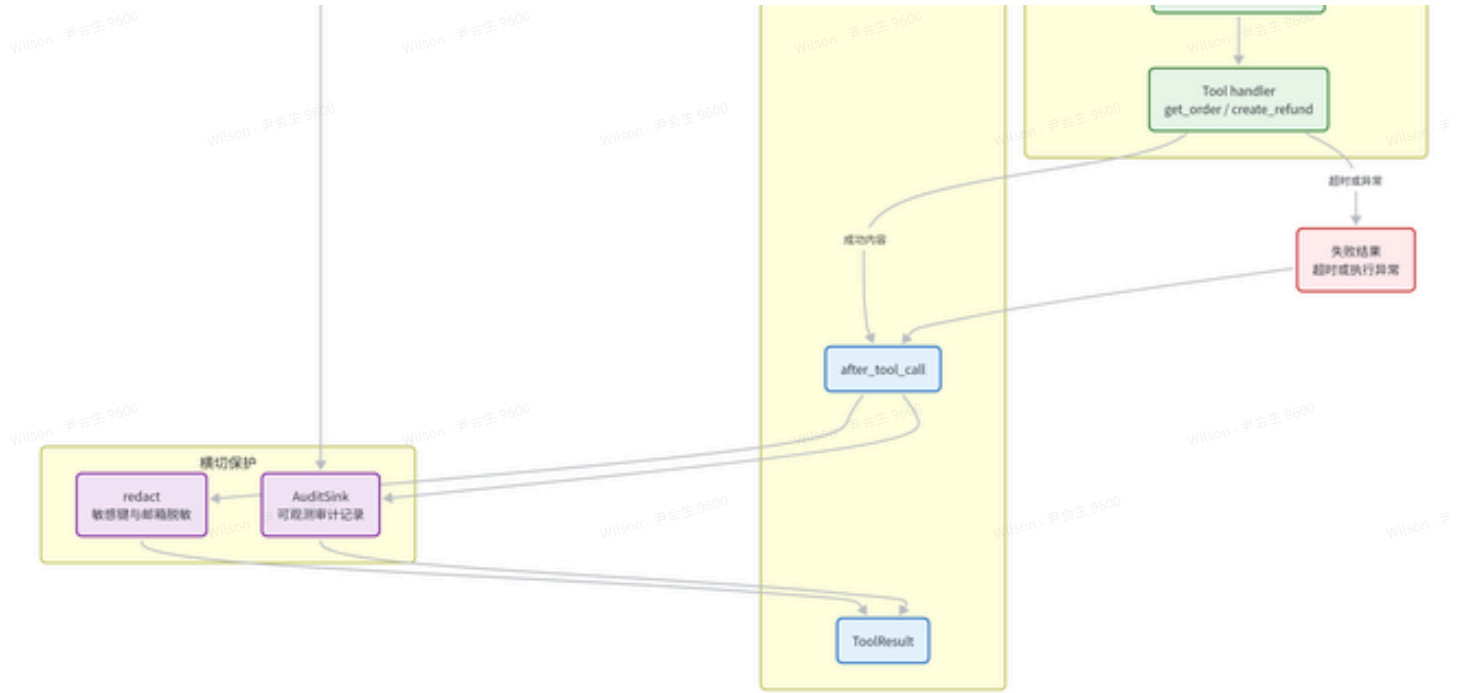
第四条（参数伪造被拒绝）

第五条（Bypass 模式下仍被拒绝）

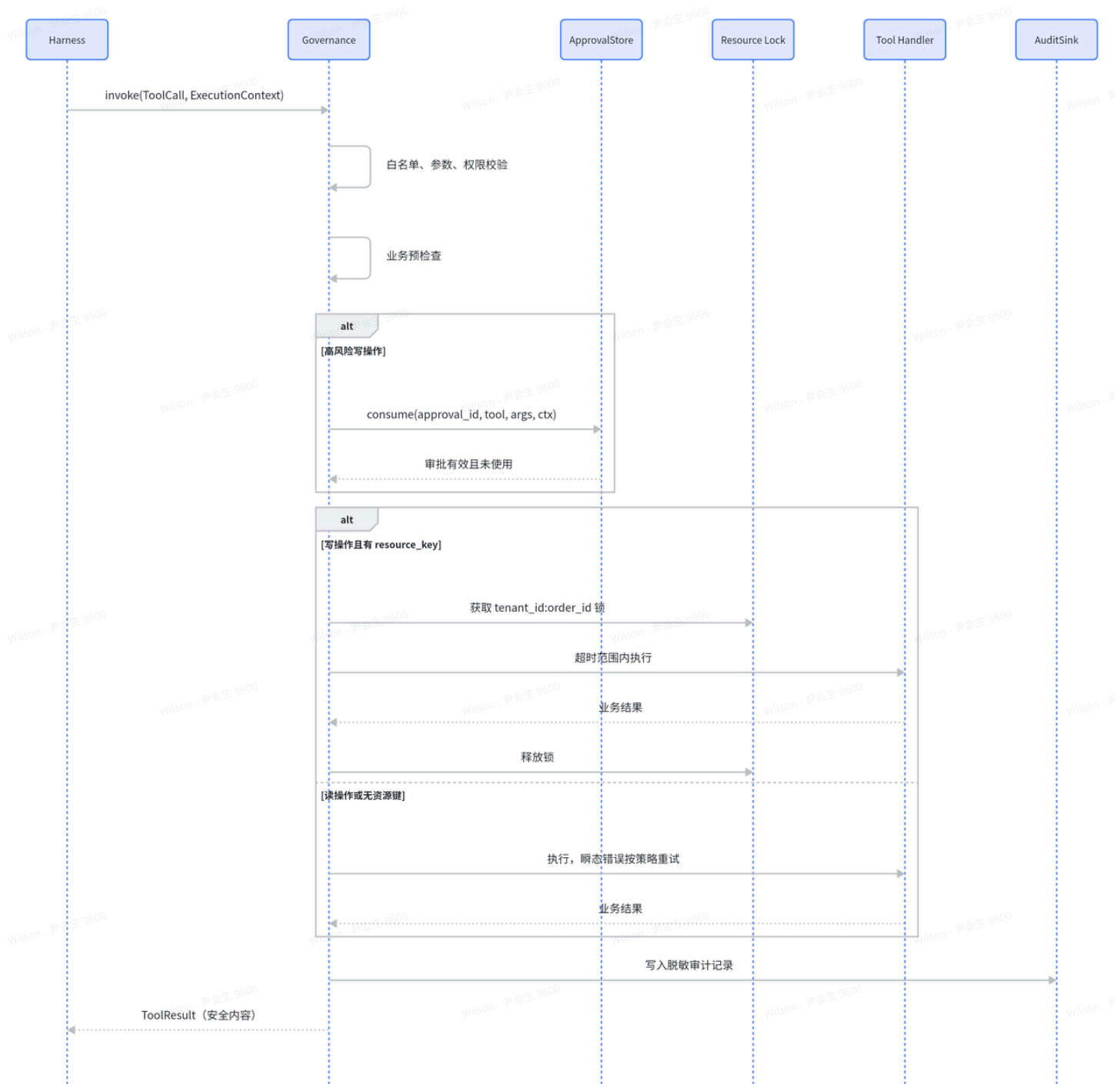
第六条（Plan 模式下写操作被拒绝）

架构与调用流程





核心路径图



2.2 模型视角 vs 开发者视角

对比维度	模型视角（AI 消费）	开发者视角（工程师排障）
可以外包给 AI	工具选择、参数填充、错误码→重试策略、结果语义理解	日志摘要生成、异常模式初步分类、关联 Trace 的排查建议
必须工程师掌控	—（模型不应触碰的已在上方“看不到”列）	审计记录写入、决策最终归因、Runtime 契约变更、敏感数据访问权限

模型只消费“经过治理的结果”，开发者看到“完整的决策过程”。两者的数据管道必须隔离。

三、人的责任和掌握标准

Agent 开发工程师的价值：把不确定的动作申请压进确定的工程边界。

3.1 交付能力：五类可验收的产出

交付能力	具体产物	验收证据
工具风险建模	ToolPolicy、读写分类、风险等级、权限与审批要求	策略表与代码一致，边界案例有说明
可信上下文设计	ExecutionContext、身份透传、租户与权限来源	模型参数无法伪造身份，跨租户测试被拒绝
确定性执行治理	唯一 ToolRuntime.invoke()、三态决策和固定优先级	deny、plan、白名单、RBAC 和审批顺序可测试
副作用与恢复控制	超时、重试、幂等键、队列和人工兜底	写操作不盲重试，未知状态可追踪
结果与证据治理	Tool Result、脱敏、稳定错误码和审计记录	敏感信息零泄漏，Trace 与 Tool Call 可关联

3.2 AI 的边界：哪些可以辅助，哪些必须人决定

AI 可以辅助：生成 Pydantic Model、dataclass、序列化、错误映射、测试样板、重构审计与脱敏

必须人决定：

- 业务建模、风险判断、责任归属
- 拒绝策略、审批语义、超时恢复策略
- 策略顺序、审计保留策略

3.3 四个控制阶段

- 1. 工具分级与权限模型（谁可以申请什么动作）
- 2. 调用期安全与韧性（允许执行以后怎样避免失控）
- 3. 调用编排与队列（多个工具如何并行/串行/排队）
- 4. 审计与可观测（怎样证明系统为什么这样决定）

四、第一阶段：工具分级与权限模型

要在动手做事之前，就先把好关。

问题还原：

`queryOrderDetailInfo` 查订单详情，Agent工程师最怕的是：A用户的Token被模型意外泄露或透传，结果B用户通过Agent查到了A的订单。或者更隐蔽的情况是，模型在调用参数里伪造了 `user_id`，说"帮我查一下user_123的订单"，而当前Token对应的用户是user_456。这个调用该不该放行？

问题的根源是，没有区分"业务参数"和"授权事实"。门店名、商品名是业务参数，可以来自模型。但 `user_id`、`tenant_id` 是授权事实，只能来自服务端。

4.1 为什么必须先给工具分级？

最小策略表解决哪些问题：

- 1. 工具是否改变外部状态？
- 2. 读取的数据是否敏感？
- 3. 当前身份需要哪项业务权限？
- 4. 影响范围是单条资源、一个租户，还是整个环境？
- 5. 失败后能否安全重放，是否真正幂等？
- 6. 是否必须由用户或审批人确认具体参数？
- 7. 运行时使用什么超时、并发和审计策略？

示例策略表：

工具	副作用	数据敏感度	风险	权限	人工确认	自动重试
----	-----	-------	----	----	------	------

get_order	只读	中，含客户信息	中	order:read	否	瞬时失败可有限重试
create_refund	写，产生资金变化	中	高	refund:create	必须绑定订单与金额	非幂等时禁止盲重试
run_shell	取决于命令，影响面可能很大	高	中到极高	shell:run	危险动作确认或直接拒绝	默认不重试

Python 策略对象：

```
代码块
1 @dataclass(frozen=True, slots=True)
2 class ToolPolicy:
3     effect: Effect
4     risk: Risk
5     permission: Permission
6     requires_approval: bool
7     timeout_seconds: float
8     max_retries: int
9     idempotent: bool
```

4.2 白名单、RBAC 和人工确认 三种机制

按需设计，没必要全上！

解决方案：可信上下文隔离

Agent工程师在网关层解析JWT，提取出 user_id 和 tenant_id ，放进 ExecutionContext 。这个上下文贯穿整个调用链，所有权限判断都基于它。

模型调 queryOrderDetailInfo 时，参数里只有 order_id ，没有 user_id 。Runtime收到调用后，用上下文中的 user_id 去查这个订单归属，如果订单不属于当前用户，直接拒绝。

三项机制配合

机制	判断对象	可信输入	订单案例	失败后的动作
工具白名单	Agent 本轮能力面	Agent 配置、租户策略、任务策略	只读客服看不到退款工具	本轮不应继续尝试

RBAC	当前主体资格	鉴权系统生成的角色与权限	普通客服没有 refund:create	换有权限主体或结束
人工确认	一次具体动作	服务端审批记录	用户确认 ord_1001 退款 399 元	暂停并进入确认流程

4.2.1 工具白名单：先限制 Agent 本轮拥有什么能力

白名单必须出现两次：

- 发现期：Harness 只把必要的工具 Schema 放进模型上下文

代码块

```
1 def model_tools(self, context: ExecutionContext) -> list[dict[str, Any]]:
2     return [
3         tool.to_model_tool()
4         for tool in self._tools.values()
5         if tool.name in context.allowed_tools
6     ]
```

- 执行期：ToolRuntime.invoke() 内重新检查 allowed_tools

代码块

```
1 if tool.name not in context.allowed_tools:
2     return PermissionDecision(
3         DecisionAction.DENY,
4         "TOOL_NOT_ALLOWED",
5         "工具不在本轮执行白名单",
6         "whitelist",
7     )
```

4.2.2 RBAC 检查（使用细粒度权限，而非角色名称）

代码块

```
1 if tool.policy.permission not in context.permissions:
2     return PermissionDecision(
3         DecisionAction.DENY,
4         "PERMISSION_DENIED",
5         f"缺少业务权限 {tool.policy.permission}",
```

```
6         "rbac",
7     )
```

4.2.3 人工确认流程

代码块

```
1  Tool Call
2  → PermissionEngine 返回 confirm
3  → Agent Loop 暂停
4  → 可信界面展示动作
5  → 用户确认并生成 approval_id
6  → 原参数重新进入 Runtime
7  → 审批完全匹配才 allow
```

4.3 身份透传时

工具白名单在Harness层发挥作用。客服的Agent配置只给 `order:read` 权限，就算Prompt注入喊"调用退款"，模型上下文中压根没有 `cancelOrder` 的Schema，想调也无从下手。

RBAC权限检查在Runtime层兜底。即使模型绕过白名单传入了 `cancelOrder` 的tool_call，Runtime检查权限集，没有 `order:cancel` 就直接拒绝。

人工确认在业务层把关。 `createOrder` 必须等待前端用户点击"确认支付"生成 `approval_id`，模型输出的参数必须与 `approval_id` 绑定的参数哈希一致，否则拒绝。

可信上下文（由 Web 鉴权中间件创建）：

代码块

```
1  @dataclass(frozen=True, slots=True)
2  class ExecutionContext:
3      trace_id: str
4      user_id: str
5      tenant_id: str
6      mode: PermissionMode
7      permissions: frozenset[Permission]
8      allowed_tools: frozenset[str]
9      approval_id: str | None = None
```

模型提供业务候选参数（订单号、金额），不能提供授权事实（`user_id`、`tenant_id`、`approved`）。

4.4 怎样把策略接入 loop？

提示词生成样板：

代码块

- 1 请只生成 Python 数据模型与接口，不实现业务 handler；
- 2
- 3 1. 使用 Python 3.12、`dataclass(frozen=True, slots=True)` 和 Pydantic 2；
- 4 2. `ToolDefinition` 包含 name、description、Pydantic 参数模型、ToolPolicy、handler、precheck 和 canonical_target；
- 5 3. `ExecutionContext` 包含服务端可信的 trace_id、user_id、tenant_id、permissions、allowed_tools、mode 和 approval_id；
- 6 4. `to\_model\_tool()` 只能返回名称、描述和 JSON Schema，禁止暴露 handler、权限和审批；
- 7 5. 给出类型注解，不要实现 PermissionEngine 和 ToolRuntime；
- 8 6. 最后解释模型侧对象与运行侧对象的边界。

工具定义：

代码块

```
1 @dataclass(frozen=True, slots=True)
2 class ToolDefinition:
3     name: str
4     description: str
5     parameters_model: type[StrictArgs]
6     policy: ToolPolicy
7     handler: Handler
8     canonical_target: CanonicalTarget
9     precheck: Precheck | None = None
10
11     def to_model_tool(self) -> dict[str, Any]:
12         return {
13             "type": "function",
14             "function": {
15                 "name": self.name,
16                 "description": self.description,
17                 "parameters": self.parameters_model.model_json_schema(),
18             },
19         }
```

所有调用必须经过同一入口：

代码块

```
1 result = await runtime.invoke(  
2     ToolCall("call_01", "get_order", {"order_id": "ord_1001"}),  
3     context,  
4 )
```

4.5 权限模式怎么设计？

模式	业务语义	仍然不可跳过的边界
default	普通交互，必要时返回confirm	deny、白名单、RBAC、业务检查、审批
plan	只允许分析和读取，不允许真实写入	所有写操作与 Shell 都被执行层拒绝
bypassPermissions	跳过普通低风险确认，便于受控环境自动化	deny、plan、白名单、RBAC、业务检查、强制审批
dontAsk	无法与用户交互	需要确认的动作直接 deny，不得偷偷执行

正确顺序：

硬拒绝(deny) → plan → 白名单 → RBAC → 业务预检查 → 强制审批 → bypass → 普通 allow

`user_id` 从网关的JWT解析，`tenant_id` 从会话上下文获取，`permissions` 从IAM系统查询。模型参数只允许传 `shop_id`、`product_id`、`size` 这些业务候选字段。身份和授权事实全部来自服务端的 `ExecutionContext`，模型说的一概不信。

4.6 本阶段小结

第一阶段真正训练的是四项判断能力：

#	能力
1	根据副作用、数据敏感度和恢复成本给工具分级，而非根据名字猜风险
2	把模型生成的业务候选参数与服务端授权事实分离
3	理解白名单、RBAC、审批和权限模式各自保护什么边界
4	用 Handler 调用次数、数据库写入或支付记录证明拒绝发生在副作用之前

五、第二阶段：调用期安全

问题还原：

`createOrder` 确认下单，真金白银扣钱。网络一卡顿，Agent的HTTP客户端超时抛异常，捕获异常后自动重试三次。三次重试可能都"成功"了——瑞幸收到了三笔订单，用户的账户被锁了三笔钱。更糟的是，用户以为第一笔就成功了，后面两笔完全不知情。

问题的根源是，没有区分"读操作"和"写操作"的重试语义。读操作查不到可以再查一次，写操作重放一次就是一次新的资金变动。

这一阶段集中解决五类运行期问题：

运行期问题	本阶段的控制点	如果不处理会发生什么
模型参数结构合法但业务含义非法	Schema 校验 + 业务预检查	非法金额、跨租户资源或过期状态进入 handler
SQL / Shell 的表达能力远大于业务需要	能力收窄 + 参数化 + 隔离	一条工具调用变成任意数据库或系统权限
下游超时、限流或断连	按副作用和幂等语义恢复	重复退款、无限重试或把未知状态当失败
工具原始结果包含敏感数据和内部异常	结果投影、脱敏、限长和错误归一化	Token、个人信息和堆栈进入模型或日志
Prompt 注入诱导模型调用危险工具	Prompt 降概率 + Runtime 硬边界	模型把外部数据中的指令变成真实副作用

5.1 参数校验不只是检查 JSON 能否解析

以下几段参数都是合法 JSON，但都不应进入退款 Handler：

代码块

```
1 {"order_id":"ord_1001","amount":-1,"reason":"退款"}
2 {"order_id":"../../etc/passwd","amount":399,"reason":"商品存在质量问题"}
3 {"order_id":"ord_1001","amount":99999999,"reason":"商品存在质量问题"}
4 {"order_id":"ord_1001","amount":399,"reason":"商品存在质量问
   题","user_id":"admin","approved":true}
```

Pydantic Model 示例：

代码块

```
1 class StrictArgs(BaseModel):
2     model_config = ConfigDict(extra="forbid", strict=True)
3
4
5 class CreateRefundArgs(StrictArgs):
6     order_id: str = Field(pattern=r"^ord_[0-9]{4}$")
7     amount: float = Field(gt=0, le=10_000)
8     reason: str = Field(min_length=4, max_length=200)
```

Runtime 校验：

代码块

```
1 try:
2     arguments = tool.parameters_model.model_validate(call.arguments)
3 except ValidationError as error:
4     return ToolResult(
5         tool_call_id=call.tool_call_id,
6         tool_name=call.name,
7         ok=False,
8         action=DecisionAction.DENY,
9         code="INVALID_ARGUMENT",
10        content=normalize_validation_errors(error),
11    )
```


5.2 Schema 正确以后，为什么还要做业务预检查？

层次	判断内容	数据来源	失败示例
Schema	字段、类型、格式和范围	ToolDefinition	金额为负、未知字段、订单号格式错误
业务预检查	资源归属、当前状态、额度和依赖事实	Repository / 业务 API	订单不属于租户、已退款、可退金额不足
权限	当前主体是否有资格申请	ExecutionContext / IAM	普通客服没有退款权限
审批	用户是否确认当前具体动作	审批服务	金额改变、审批过期、审批已使用

业务预检查示例：

代码块

```
1  async def refund_precheck(arguments: ArgsModel, context: ExecutionContext) ->
    None:
2      assert isinstance(arguments, CreateRefundArgs)
3      order = ORDERS.get((context.tenant_id, arguments.order_id))
4
5      if not order or order["status"] != "paid":
6          raise PolicyDenied("BUSINESS_RULE_DENIED", "订单不存在或状态不可退款")
7
8      if arguments.amount > float(order["refundable"]):
9          raise PolicyDenied("BUSINESS_RULE_DENIED", "退款金额超过可退金额")
```

5.3 SQL 和 Shell 为什么不能原样交给模型？

安全做法：把能力收窄成业务工具。

不要给模型：

代码块

```
1  run_sql({"sql": "模型生成的任意 SQL"})
2  run_shell({"command": "模型生成的任意命令"})
```

而应该：

代码块

```
1  async def search_orders(  
2      pool: DatabasePool,  
3      context: ExecutionContext,  
4      status: Literal["paid", "shipped", "refunded"],  
5      limit: int,  
6  ) -> List[dict[str, Any]]:  
7      safe_limit = max(1, min(limit, 50))  
8      return await pool.fetch(  
9          """  
10         SELECT order_id, status, total_amount  
11         FROM orders  
12         WHERE tenant_id = $1 AND status = $2  
13         ORDER BY created_at DESC  
14         LIMIT $3  
15         """,  
16         context.tenant_id,  
17         status,  
18         safe_limit,  
19     )
```

Shell 同理，固定可执行文件和参数数组，不使用 `shell=True`。

代码块

```
1  async def inspect_service_logs(service: str, requested_lines: int) -> str:  
2      service_units = {"api": "my-api.service", "worker": "my-worker.service"}  
3      unit = service_units[service]  
4      lines = max(1, min(requested_lines, 200))  
5      process = await asyncio.create_subprocess_exec(  
6          "journalctl",  
7          "--unit",  
8          unit,  
9          "--lines",  
10         str(lines),  
11         "--no-pager",  
12         stdout=asyncio.subprocess.PIPE,  
13         stderr=asyncio.subprocess.PIPE,  
14     )  
15     stdout, _ = await asyncio.wait_for(process.communicate(), timeout=5)  
16     return stdout.decode(errors="replace")[:20_000]
```

5.4 超时以后为什么不能统一重试三次？

区分机制：

机制	发生位置	是否产生新的模型决策	是否重放同一业务动作	典型场景
Provider / 大模型请求重试	调用 LLM API 时	否，通常还是同一请求	尚未进入 handler	429、连接失败、模型服务 5xx
Agent 重新规划	Tool Result 回填后的下一轮	是，会生成新的 Tool Call	不一定，参数和工具都可能变化	INVALID_ARGUMENT 后模型修正金额
Tool Runtime 重试	execute 阶段	否，沿用同一 Tool Call	是，因此必须判断幂等性	只读查询遇到瞬时数据库断连
业务状态恢复	结果未知或长流程中断后	不一定	原则上先查状态，不直接重放	支付请求超时后查询退款单

恢复策略：

失败	工具属性	推荐动作	禁止动作
参数错误	任意	返回 INVALID_ARGUMENT ，允许模型修参	用原参数重试
权限或 deny 拒绝	任意	结束、换主体或人工处理	让模型改写提示词绕过
只读瞬时失败	read	总 deadline 内指数退避、有限重试	无限重试占满资源
幂等写失败	write + 真实幂等	使用同一幂等键有限重试	每次生成新幂等键
非幂等写超时	write	返回 TIMEOUT_UNKNOWN ，查询真实状态	直接重放
依赖持续不可用	任意	缓存、只读降级或人工兜底	伪造成功结果

Python Runtime 先计算允许重试的次数：

代码块

```
1  retries = (  
2      tool.policy.max_retries  
3      if tool.policy.effect is Effect.READ or tool.policy.idempotent  
4      else 0  
5  )
```

随后使用 `asyncio.timeout()` 包住 handler:

```
代码块  
1  for attempt in range(retries + 1):  
2      try:  
3          async with asyncio.timeout(tool.policy.timeout_seconds):  
4              return await tool.handler(tool_call_id, arguments, context)  
5      except TransientToolError:  
6          if attempt == retries:  
7              raise  
8          await asyncio.sleep(min(0.05 * (2**attempt), 0.2))
```

解决方案：副作用驱动的重试策略

`createOrder` 是非幂等的写操作，`max_retries` 设为0，禁止自动重放。超时后返回错误码 `TIMEOUT_UNKNOWN`，不抛异常让上层去猜。

Agent Loop收到 `TIMEOUT_UNKNOWN` 后，强制调用 `queryOrderDetailInfo` 查单接口。查到了就继续，查不到再报错人工介入。绝对不能在未知状态下盲目重试。

5.5 原始结果：不能同时发给模型、用户和日志

视图	目的	可以包含	不应默认包含
handler 原始结果	内部业务处理	下游完整对象	不直接传播
模型视图	支撑下一步推理	状态、可退金额、业务 ID	Token、密码、内部堆栈、无关个人信息
用户视图	告知结果与下一步	用户有权查看的字段	内部权限规则与系统实现细节
审计视图	证明决策与执行	Trace、Tool Call、主体、错误码、摘要	完整 Prompt、原始客户资料、密钥

脱敏函数：

代码块

```
1 def _redact(value: Any) -> Any:
2     if isinstance(value, Mapping):
3         return {
4             key: "***"
5             if re.search(r"token|secret|password|authorization", key, re.I)
6             else _redact(item)
7             for key, item in value.items()
8         }
9     if isinstance(value, str):
10        return re.sub(EMAIL_PATTERN, "***@***", value)
11    return value
```

5.6 Prompt 注入：最终靠工具边界兜底

Prompt 注入治理的目标：不是证明模型永远不会受骗，而是保证模型即使受骗，也只能提出候选动作，不能绕过 Runtime 获得真实权限。

- Prompt 层：明确区分指令和不可信数据
- 数据层：对网页、邮件、Tool Result 做来源标记、限长、内容隔离
- Runtime 层（最终边界）：所有 Tool Call 重新经过白名单、RBAC、资源检查、审批

5.7 隔离环境解决什么，权限判断又解决什么？

- 权限判断：这次动作应不应该做（业务正当性）
- Sandbox / 隔离：即使代码失控，最多能伤到哪里（技术爆炸半径）

两者不能互相替代。

5.8 本阶段小结：韧性不是多重试几次

这一阶段我们扩展了四项工程判断能力。

第一，能够区分 JSON 可解析、Schema 合法、业务合法、权限允许和用户确认。

第二，优先把 SQL 和 Shell 收窄成具体的业务能力，然后再用 AST、沙箱和最小权限来防御剩余的风险。

第三，根据只读、幂等和真实的副作用来选择重试、查状态、降级还是人工接管。

第四，在数据进入模型、用户界面和审计之前，先控制好结果的视图。

下一节我们要讨论的是队列与编排——重点不是一上来就建 DAG，而是先识别哪些调用是独立的、哪些有依赖关系、哪些会争抢同一份资源。

六、第三阶段：调用编排与队列 (了解即可)

工具队列要先判断调用是否独立、是否存在数据依赖、是否争用同一业务资源，再选择有限并行、显式串行或按资源键排队。

问题还原：

`CancelOrder` 取消订单或退款，用户说"帮我取消最近一单"。模型查到了最近一单，调了 `cancelOrder` 接口，但瑞幸返回了报错——因为那杯咖啡五分钟前已经被取餐了。更严重的情况是，如果接口设计不严谨，取餐后的取消请求可能引发资金异常。

问题的根源是，没有在Handler执行前做业务状态预检查。模型只知道"用户想取消订单"，但不知道"这个订单当前状态是否允许取消"。

解决方案：业务预检查

在 `cancelOrder` 的Handler执行之前，先走业务预检查。查询订单状态，如果状态是 `picked_up` 或 `completed`，直接返回 `BUSINESS_RULE_DENIED`，Handler连执行的机会都没有。

业务预检查是独立的一层，在权限检查之后、Handler执行之前。它的数据来源是Repository或业务API，不依赖模型的任何判断。

6.1 先识别三种关系

调用关系	判断依据	推荐方式	订单案例
相互独立	输入完整，结果互不依赖	有上限地并行	同时查订单、物流、优惠券
数据依赖	后一步参数来自前一步结果	显式串行	先查可退金额，再退款
资源冲突	多个动作修改同一资源	按资源键串行	同一订单的两笔退款

6.2 结果完成顺序和消息顺序为什么要分开？

代码块

```
1  async def invoke_many(calls, context):
2      tasks = {
3          asyncio.create_task(runtime.invoke(call, context)): call.tool_call_id
4          for call in calls
5      }
6      by_id = {}
7      for task in asyncio.as_completed(tasks):
8          result = await task
9          by_id[result.tool_call_id] = result
10     return [by_id[call.tool_call_id] for call in calls]
```

- 每个结果携带自己的 ID
- 聚合器不用“第几个完成”猜“第几个调用”

6.3 什么时候才需要 DAG 或 workflow 引擎？

只有业务出现稳定的多步依赖、条件分支、长时间等待和补偿，才值得升级为显式 Graph。

选择标准：是否需要持久化进度、跨故障恢复和补偿语义。

下单链路必须串行执行：查店、搜品、预览、等待确认、下单。每一步的结果是下一步的前提。如果模型先调 `createOrder` 再调 `queryShopList`，Runtime 应该直接拒绝，因为顺序本身就是错误的。

七、第四阶段：审计与可观测——怎样证明系统为什么这样决定？

审计是在不扩大数据泄漏的前提下，记录足以重建一次治理决策和业务副作用的证据。

问题还原

问题一，查不到门店死循环。`queryShopList` 查附近门店，模型把“南京东路”传成了“南京西路”，瑞幸返回了空结果。模型收到空结果后，不知道是“参数错了”还是“真的没有门店”，用同样的

参数反复重试，陷入死循环，浪费token、消耗API配额、用户干等。根源是Tool Result只返回了业务数据（空列表），没有返回稳定错误码，模型无法区分不同失败场景。

问题二，事故来了查不清。某天早上，客服收到用户投诉："我根本没有下过这单，为什么扣了我的钱？"产品经理要求提供完整的操作记录。Agent工程师打开日志，只看到零散的HTTP请求记录，根本拼不出完整画面：用户当时说了什么？模型生成了什么Tool Call？经过了哪些决策环节？是模型自己决定还是用户确认的？用的是谁的Token？每一笔订单都是"从天而降"，每一笔退款都是"凭空消失"，对账对不上、追责追不到、事故复盘全靠猜。

7.1 一条最小审计记录应该包含什么？

生成本节审计对象的提示词：

代码块

```
1 请只生成 Python 3.12 的 `AuditRecord` 数据类与内存 `AuditSink`，不要生成 Runtime。
2
3 要求：
4 1. 记录 `trace_id`、`tool_call_id`、`tool_name`、`user_id`、`tenant_id`；
5 2. `phase` 只能是 `decision` 或 `execution`；
6 3. 保存稳定 `decision`、`code`、参数键名和可选延迟，不默认保存参数值；
7 4. 对象不可变，适合 pytest 精确断言；
8 5. `AuditSink` 只提供追加与按 trace 查询，不打印 Prompt、Token、密码或原始异常；
9 6. 给出两个测试：被拒绝的调用只有 decision 记录，成功调用同时有 decision 与 execution
   记录。
10
11 先输出接口说明，再输出最小代码；不得替我决定审计保留期限和敏感字段分类。
```

生成的代码：

代码块

```
1 @dataclass(frozen=True, slots=True)
2 class AuditRecord:
3     trace_id: str
4     tool_call_id: str
5     tool_name: str
6     user_id: str
7     tenant_id: str
8     phase: Literal["decision", "execution"]
9     decision: str
10    code: str
11    argument_keys: tuple[str, ...]
```


12 latency_ms: int | None = None

- 只保存 `argument_keys`，不默认保存全部参数值
- `trace_id` 串起一次 Agent Run
- `tool_call_id` 关联模型 Tool Call 与 Tool Result
- 业务 ID（订单号、退款单号）核对真实副作用

7.2 为什么决策审计和执行审计要分开？

- decision：记录 allow、deny 或 confirm，以及规则来源和错误码
- execution：记录 executed 或 failed，以及延迟和业务结果摘要

两阶段记录让我们知道“是否获准”和“是否完成”。

7.3 稳定错误码怎样驱动 Agent Loop？

错误码	Agent Loop / Harness 动作	是否自动重试
<code>INVALID_ARGUMENT</code>	把字段错误回填模型，允许生成新的 Tool Call	不用原参数重试
<code>TOOL_NOT_ALLOWED</code>	当前轮不开放，结束该路径	否
<code>PERMISSION_DENIED</code>	当前身份无权限，换主体或结束	否
<code>APPROVAL_REQUIRED</code>	暂停 Loop，进入可信确认界面	否
<code>BUSINESS_RULE_DENIED</code>	业务状态不允许，重新规划或人工处理	通常否
<code>TIMEOUT</code>	只读或幂等工具按策略恢复	有条件
<code>TIMEOUT_UNKNOWN</code>	查询真实业务状态，禁止直接重放	否
<code>TOOL_ERROR</code>	记录内部诊断，给模型有限说明	按映射决定

7.4 审计怎样避免自己成为泄漏源？

- 参数先按字段分类，默认只记键名和摘要

- 2. 模型结果与审计结果复用同一套敏感字段识别规则
- 3. 内部 traceback 进入受保护的诊断通道
- 4. 审计存储设置访问控制、保留期限和不可篡改策略
- 5. 高风险事件保留审批引用与业务 ID

7.5 怎样用一条事件时间线复盘工具调用？

代码块

```
1  10:00:00.120  model_tool_call  call_03 create_refund
2  10:00:00.123  schema_valid     call_03
3  10:00:00.127  policy_allow     call_03 code=APPROVED
4  10:00:00.130  tool_start       call_03
5  10:00:00.286  tool_end         call_03 refund_id=ref_9001
6  10:00:00.288  result_redacted  call_03
7  10:00:00.291  tool_message     call_03
```

如果调用被拒绝，时间线应停在 `policy_deny`，不应出现 `tool_start`。

7.6 审计指标怎样反过来改进治理策略？

- `INVALID_ARGUMENT` 比例 → Schema 是否易用
- `TOOL_NOT_ALLOWED` 比例 → 任务装配或工具描述问题
- `APPROVAL_REQUIRED` 到批准转化率 → 确认流程是否过度
- `PERMISSION_DENIED` 分布 → 角色配置是否匹配
- `TIMEOUT_UNKNOWN` 数量 → 写接口是否缺少状态查询和幂等设计
- 敏感字段命中次数 → 下游是否返回过多数据

7.7 审计数据保留多久、谁能查看？

- 高风险写操作：长期保留决策码、主体、审批引用和业务 ID
- 普通只读调用：保留聚合指标或短期诊断记录
- 敏感字段默认不采集
- 访问审计本身也应被审计

解决方案一：稳定错误码驱动Agent Loop

`queryShopList` 查不到门店时，返回的错误码是 `SHOP_NOT_FOUND`，而不是一个自然语言的"抱歉没找到"。Agent Loop收到 `SHOP_NOT_FOUND` 后，知道是门店名不对，触发模型重新生成门店名参数再查一次。如果还是查不到，就报错让用户确认门店名，而不是无限重试。

各种错误码各司其职。`INVALID_ARGUMENT` 来了，Agent知道是参数格式错了，重新生成参数。`PERMISSION_DENIED` 来了，Agent知道当前身份不行，换主体或结束，不会用相同参数再试一次。`TIMEOUT_UNKNOWN` 来了，强制调查单接口确认状态，不直接重放。`APPROVAL_REQUIRED` 来了，暂停Loop等用户确认。

Agent工程师切记，不要让Agent去解析"抱歉，门店已关闭"这种自然语言来判断下一步。用稳定错误码驱动Loop，每一步都是确定性的。

解决方案二：审计记录构建完整证据链

Agent工程师需要一条完整的证据链，从用户输入到工具执行，每一步都可追溯。

最小审计记录包含五层信息。`trace_id` 串联整次对话——用户从"查门店"到"确认下单"的所有操作都在同一条trace下，一笔订单对应一个trace。`tool_call_id` 关联模型请求——Agent Loop里每个Tool Call都有唯一ID，审计记录里的 `tool_call_id` 和Tool Result里的 `tool_call_id` 是同一个，用户确认时传的 `approval_id` 也能关联回来。决策码说明是allow、deny还是confirm——拒绝了的调用也要记，证明"不该执行的确实没执行"。规则来源说明是白名单、RBAC、幂等还是业务预检——同一笔订单被拦截了，能查清楚是哪条规则拦的。参数摘要存SHA256而不存明文——门店名、商品名、订单号这些业务字段不直接存日志，存哈希值用于对账，Token和手机号永远不进审计。

7.8 本阶段小结

审计与可观测是一条从模型申请、治理决策、Handler 执行到 Tool Result 回填的结构化证据链。

它解决了三个问题：

被拒绝的动作能不能证明副作用为零？

成功或超时的动作能不能关联到真实业务结果？

线上的策略能不能依据数据改进？

没有这些，系统就无法对账，也无法判断治理代码是否真正有效。

我们做了五件事：

定义最小 AuditRecord，用 trace_id、tool_call_id 和业务 ID 关联不同层次；

把 decision 和 execution 分开记录；

用稳定错误码驱动 Agent Loop；

对参数、结果和异常执行最小披露；

最后把事件组织成时间线、指标和人工恢复入口。

下一节，我们把四段压回一条 ToolRuntime.invoke()。

八、把四个阶段接回 Runtime

治理代码只有放在所有调用方都无法绕过的唯一入口里，才是真正的安全边界。

Agent 生命周期	Python v11 对应	治理责任
Tool Schema 校验	Pydantic <code>model_validate()</code>	把不可信字典变成业务对象
<code>beforeToolCall</code>	<code>PermissionEngine.decide()</code>	白名单、RBAC、业务预检查、审批和风险
<code>AgentTool.execute</code>	<code>ToolDefinition.handler</code>	只实现已经通过治理的业务动作
执行模式与取消	<code>asyncio.timeout</code> 、重试策略、队列	控制期限、重放和并发
<code>afterToolCall</code>	finalize 脱敏与 <code>AuditSink</code>	结果进入模型以前收口
Tool Result Message	<code>ToolResult.to_tool_message()</code>	保留 <code>tool_call_id</code> 回填 Agent Loop

生成 `ToolRuntime.invoke()` 骨架的提示词：

代码块

- 1 请基于现有 Python 3.12 接口生成 `ToolRuntime.invoke()` 的最小骨架，严格分为 prepare、execute、finalize 三段。
- 2
- 3 现有对象：`ToolCall`、`ToolResult`、`ToolDefinition`、`ExecutionContext`、`PermissionEngine`、`AuditSink`。

```
4
5 硬约束：
6 1. 所有入口只能调用 `invoke()`，不得在 Agent Loop 中直接访问 handler；
7 2. prepare 顺序为：查找工具、Pydantic 校验、权限三态决策；
8 3. `DENY` 和 `CONFIRM` 必须立即返回结构化 ToolResult，handler 零调用；
9 4. execute 只接收已校验的 Pydantic 对象，超时与重试委托 `_execute_with_recovery()`；
10 5. finalize 对结果投影、脱敏、审计，并原样保留 `tool_call_id`；
11 6. Python 异常不得直接进入模型消息；
12 7. 只生成 Runtime 组装代码和三项最小测试，不修改 PermissionEngine 的业务规则。
13
14 输出后逐项说明每段位于生命周期哪里、交换顺序的具体后果。
```

Runtime 骨架：

代码块

```
1  async def invoke(self, call: ToolCall, context: ExecutionContext) ->
    ToolResult:
2      # prepare: 找工具、参数校验、权限三态决策
3      tool = self._tools.get(call.name)
4      arguments = tool.parameters_model.model_validate(call.arguments)
5      decision = await self._permission_engine.decide(tool, arguments, context)
6      if decision.action is not DecisionAction.ALLOW:
7          return blocked_result(call, decision)
8
9      # execute: 按副作用、超时和幂等策略运行 handler
10     raw = await self._execute_with_recovery(tool, call.tool_call_id,
        arguments, context)
11
12     # finalize: 投影、脱敏、审计，并保留 tool_call_id
13     return ToolResult(
14         tool_call_id=call.tool_call_id,
15         tool_name=call.name,
16         ok=True,
17         action=DecisionAction.ALLOW,
18         code="OK",
19         content=redact(raw),
20     )
```

模型模式的 Agent Loop 最终调用：

代码块

```
1  result = await runtime.invoke(
2      ToolCall(
3          tool_call_id=provider_call.id,
```

```

4         name=provider_call.function.name,
5         arguments=json.loads(provider_call.function.arguments),
6     ),
7     context,
8 )
9 messages.append(result.to_tool_message())

```

8.1 最小规格、Harness、TDD 和 Loop 怎样组合？

1. 最小规格：工具策略表、稳定错误码、验收清单（只记录硬性要求）
2. Harness：ExecutionContext、发现期白名单、PermissionEngine、唯一 Runtime、超时与结果收口
3. TDD：安全属性和副作用测试（错误码 + 副作用计数）
4. Loop：每次只接入一条规则，经历失败测试、最小修复、全量回归

8.2 迁移到真实项目时需要替换什么？

演示对象	生产替换	必须保留的硬性要求
ORDERS 字典	带 tenant 条件的 Repository 或业务 API	资源查询必须使用服务端租户
ApprovalStore	审批数据库、工作流服务或可信确认结果	绑定用户、租户、工具、参数、过期和一次性
SIDE_EFFECTS 计数	Mock 支付网关、测试数据库、消息记录	拒绝时真实副作用为零
AuditSink.records	append-only 审计表、事件流或日志平台	决策与执行可关联，不泄漏敏感数据
模拟退款 handler	支付接口	幂等键、超时、状态查询和对账
递归正则脱敏	字段投影、数据分类、统一 Masking Middleware	进入模型和普通日志前已处理
本地函数 Tool	内部 HTTP 或 MCP Tool	所有执行仍走同一个 invoke

8.3 本阶段小结

Runtime 是所有工具执行的控制平面，它消除入口漂移和检查顺序漂移。用 prepare → execute → finalize 固定生命周期，并让所有调用方共用 invoke()。

下一节将进一步展开整个系统最关键的部分：怎样把附件中的权限流程图和参考实现，翻译成一段顺序不可交换的状态机。

九、怎样让 AI 生成这套 Python 治理代码？

完整工程不是一次生成的。我们推荐把代码分成五个可验证的产物，每个产物都需要明确分工：

产物	生成前人必须决定	AI 适合补全	本轮验收
对象与 Schema	模型字段、可信字段、错误协议	数据类、Pydantic Model	伪造字段被拒绝
PermissionEngine	决策表、优先级、审批语义	分支与规则匹配样板	deny/confirm 副作用为零
ToolRuntime	prepare/execute/finalize 顺序	统一入口与异常映射	所有入口无法旁路
测试与故障注入	业务硬性要求、副作用证据	pytest 夹具与参数化	删除护栏时测试会红
Provider 组装	消息协议、Tool Call/Result 关联	DeepSeek Adapter 与流式解析	真实模型仍调用同一 invoke

下面这份提示词保留了“完整 Agent Loop + Tool Runtime”的目标，同时增加了治理状态机与离线测试。

代码块

```
1  **提示词：**
2
3  请生成一个基于 Python 3.12、Pydantic 2、pytest 和 OpenAI Python SDK 的可运行工程，实现最小但完整的 Agent 工具治理 Runtime。代码用于教学，必须清晰展示不可信 Tool Call 怎样经过确定性治理后才进入 handler。
4
5  ## 一、核心目标
6
```

实现两种运行模式：

1. 离线治理演示：不需要 API Key，直接构造 Tool Call，验证参数校验、权限、审批、执行、脱敏和审计；
2. 可选 DeepSeek Agent Loop：通过 OpenAI-compatible API 接收真实模型 Tool Call，但所有调用仍进入同一个 `ToolRuntime.invoke()`。

完整闭环为：

```
```text
用户输入
→ LLM 生成 Tool Call
→ 转换为内部 ToolCall
→ ToolRuntime.prepare
→ PermissionEngine 返回 allow / deny / confirm
→ ToolRuntime.execute
→ ToolRuntime.finalize
→ 带 tool_call_id 的 ToolResult 回填 Context
→ 下一轮 LLM 或暂停确认
```
```

二、交付文件

请输出完整文件，不使用省略号：

1. `requirements.txt`
2. `pyproject.toml`
3. `tool\_governance\_demo.py`
4. `tests/test\_tool\_governance.py`

运行命令必须是：

```
```bash
python -m pip install -r requirements.txt
python -m py_compile tool_governance_demo.py tests/test_tool_governance.py
python tool_governance_demo.py
python -m pytest -q
```
```

三、内部对象

必须实现：

- `ToolCall`：包含 `tool\_call\_id`、工具名和原始参数；
- `ToolResult`：包含原始 `tool\_call\_id`、工具名、成功状态、`allow/deny/confirm`、稳定错误码、内容和可重试语义；

- 50 - ``ToolPolicy``: 包含 ``effect``、``risk``、``permission``、``requires_approval``、超时、最大重试次数和幂等属性;
- 51 - ``ToolDefinition``: 包含名称、描述、Pydantic 参数模型、策略、handler、业务 precheck 和规范化目标函数;
- 52 - ``ExecutionContext``: 包含服务端生成的 trace、用户、租户、权限、本轮工具白名单、权限模式和审批 ID;
- 53 - ``PermissionEngine``: 只做三态决策, 不执行 handler;
- 54 - ``ToolRuntime``: 模型、CLI、离线演示和测试共用的唯一执行入口;
- 55 - ``ApprovalStore`` 与 ``AuditSink``。

56
57 模型只能看到工具名称、描述和 Pydantic 生成的 JSON Schema, 不能看到 handler、权限集合、审批记录和内部策略。

58 59 ## 四、工具与业务场景

60
61 实现三个工具:

- 62
63 - ``get_order``: 只读, 权限 ``order:read``, 返回订单状态、可退金额、客户邮箱和模拟 access token;
- 64 - ``create_refund``: 高风险写操作, 权限 ``refund:create``, 必须人工确认;
- 65 - ``run_shell``: 教学模拟 Shell, 权限 ``shell:run``, 只增加计数, 不启动真实子进程。

66
67 模型参数只能包含业务候选字段。``user_id``、``tenant_id``、角色、权限、``approved`` 和 ``approval_id`` 不得出现在工具 Pydantic Model 中, 只能来自服务端 ``ExecutionContext``。

68
69 所有参数模型继承严格基类, 使用 ``extra="forbid"`` 和严格类型。退款参数约束订单号格式、金额范围和原因长度。

70 71 ## 五、权限三态与固定顺序

72
73 定义 ``allow | deny | confirm``, 每个决策包含稳定 ``code``、安全 ``reason`` 和 ``source``。必须严格按下列顺序实现, 不得交换:

- 74
75 1. deny 规则;
- 76 2. plan 模式拒绝写操作和 Shell;
- 77 3. 执行期重新检查 ``allowed_tools``;
- 78 4. RBAC 权限;
- 79 5. 租户资源归属、状态和额度等业务 precheck;
- 80 6. 高风险或 ``requires_approval`` 工具检查一次性参数绑定审批;
- 81 7. ``bypassPermissions`` 只跳过普通确认, 不能跳过 1-6;
- 82 8. allow 规则;
- 83 9. 教学用危险 Shell 模式检测;
- 84 10. 默认允许。

85
86 deny 必须优先于 allow 和 bypass; plan 必须由执行层强制只读; 高风险业务审批不能被 allow、bypass 或会话白名单跳过。

87

六、审批要求

审批记录必须绑定：

- `user\_id`
- `tenant\_id`
- `tool\_name`
- 递归排序、稳定 JSON 序列化后的参数 SHA-256
- 过期时间
- 一次性 `used` 状态

批准 100 元后把金额改成 399 元，旧审批必须失效；同一个审批成功执行一次后不得重放。禁止使用 `approved=true`。

七、Runtime 执行要求

`ToolRuntime.invoke()` 严格分成：

1. prepare: 查找工具、Pydantic 校验、权限三态决策；
2. execute: 只有 allow 才调用 handler，根据只读/幂等属性选择超时与重试；
3. finalize: 字段投影、递归脱敏、审计并生成 ToolResult。

任何 deny 或 confirm 都必须保证 handler 零调用。参数、权限、审批和 deny 失败不得自动重试。只读或真正幂等的工具才允许有限重试；非幂等操作超时返回 `TIMEOUT\_UNKNOWN`，禁止直接重放。

八、结果与审计

返回模型以前必须脱敏邮箱以及键名包含 token、secret、password、authorization 的字段。审计至少保存：

- `trace\_id`
- `tool\_call\_id`
- 工具名
- 用户和租户
- decision / execution 阶段
- allow / deny / confirm / executed / failed
- 稳定错误码
- 参数键名
- 延迟

审计不得默认保存 Token、完整 Prompt、未经脱敏的客户资料和 Python traceback。ToolResult 必须保留原始 `tool\_call\_id`。

九、DeepSeek 可选模式

129 使用 `AsyncOpenAI`，从 `DEEPSEEK\_API\_KEY` 读取密钥；Base URL 默认
`https://api.deepseek.com`；模型从 `DEEPSEEK\_MODEL` 读取，默认 `deepseek-v4-
flash`；设置 `stream=True`，逐块输出文本并按 Tool Call index 累积流式名称与参数；通
过 `extra\_body={"thinking":{"type":"disabled"}}` 关闭思考输出。

130

131 Agent Loop 最多 8 轮。模型工具列表必须来自 Runtime 的发现期白名单；解析 Provider
Tool Call 后调用同一个 `ToolRuntime.invoke()`；Tool Result 以 `role="tool"` 和原
`tool\_call\_id` 回填。不要在 Agent Loop 中用 if/else 分发 handler。

132

133 **## 十、安全边界**

134

135 - 不执行真实 Shell，不连接真实支付接口；

136 - 演示可以用最小危险命令正则，但注释必须说明生产环境优先使用窄工具、AST、沙箱、最小权限和
fail-closed；

137 - 不允许模型声明身份、租户、权限或审批；

138 - 不把 handler 暴露给模型；

139 - 不在 handler 以后才做权限判断；

140 - 不把原始异常堆栈交给模型；

141 - 不省略 `tool\_call\_id`；

142 - 不为离线测试另造一套 Fake Runtime。

143

144 **## 十一、离线演示结果**

145

146 至少输出：

147

148 1. 查询成功且邮箱、Token 已脱敏；

149 2. 未审批退款返回 `APPROVAL\_REQUIRED` 和 confirm；

150 3. 参数绑定审批后退款成功；

151 4. 伪造 `user\_id`、`approved` 被 Pydantic 拒绝；

152 5. bypass 模式下 deny 规则仍拒绝危险 Shell；

153 6. plan 模式拒绝退款；

154 7. 最终只有一次退款副作用，Shell 副作用为零，并打印审计数量。

155

156 **## 十二、测试验收**

157

158 至少实现八项测试：

159

160 1. deny-first: bypass 不能覆盖硬拒绝；

161 2. plan: 写工具在审批与 allow 以前被拒绝；

162 3. Schema: 模型不能伪造身份和确认；

163 4. RBAC: 权限不足时 handler 零调用；

164 5. 审批改参: 批准 100 元后改成 399 元，旧审批失效；

165 6. 结果边界: 敏感字段脱敏，审计保留 tool_call_id；

166 7. 双重白名单: 模型不可见，旧 Tool Call 在执行时也被拒绝；

167 8. 一次性审批: 第一次执行成功，第二次重放返回 confirm，副作用总数为 1。

168

9.1 提示词关键内容

提示词先定义：

- 哪些对象存在，各自负责什么
- 哪些字段来自模型，哪些只能来自服务端
- 权限是三态协议
- 十条决策顺序不可交换
- 高风险审批的业务语义
- Runtime 的唯一入口与三个阶段
- 超时和重试依据副作用
- 模型、用户和审计看到不同结果
- 演示必须输出什么
- 测试必须用什么副作用证据验收

9.2 AI 生成以后应该怎样审查？

1. 对象边界：handler 是否出现在模型工具描述？身份/审批字段是否混入参数 Model？Agent Loop 是否绕过 Runtime？
2. 执行顺序：Pydantic 校验是否在 handler 之前？deny 是否先于 bypass？plan 是否由执行层检查？审批是否绑定参数且先于 allow？
3. 失败语义：写操作超时是否盲目重试？异常是否泄漏 traceback？confirm 是否被当作 allow？
4. 证据：拒绝测试是否断言 handler 零调用？脱敏测试是否同时检查模型结果和审计记录？一次性审批是否防重放？

9.3 生成代码不理想，怎么迭代？

代码块

- 1 轮次 1：只生成对象模型、Pydantic Schema 与 ToolResult
- 2 轮次 2：加入 PermissionEngine 和拒绝路径测试
- 3 轮次 3：组装唯一 ToolRuntime 与离线调用骨架
- 4 轮次 4：加入审批、超时恢复、脱敏、审计和完整测试
- 5 轮次 5：接 DeepSeek Provider，并证明没有旁路 Runtime

9.4 怎样把测试失败有效地反馈给 AI?

示例反馈：

代码块

```
1 运行 `python -m pytest -q` 后，
2 `test_approval_is_bound_to_canonical_arguments` 失败：
3 第二次调用仍执行了退款，`refund_executions == 2`。
4
5 预期：审批一次性消费，第二次返回 `APPROVAL_REQUIRED`，副作用总数为 1。
6 只允许修改 `ApprovalStore.consume()` 和相关测试夹具；
7 不得删除参数摘要校验，不得把高风险工具改成无需审批。
8 请先解释根因，再给最小补丁。
```

9.5 怎样防止 AI 通过修改测试逃避约束？

- 把业务硬性要求单独列在任务开头，声明未经批准不能改动
- 单轮只允许修改实现文件，测试变更需单独解释
- CI 中保护关键安全测试
- 检查断言是否仍包含对真实副作用的验证
- 使用变异测试——故意注入错误，证明测试真的会红

9.6 生成代码时哪些上下文应该提供，哪些不应该？

应该提供：认证中间件怎么生成 Context、支付客户端是否支持幂等键、审批服务返回什么 ID、Repository 如何带租户条件

不应该提供：生产密钥、完整客户数据、未经处理的内部日志、大段无关仓库上下文

9.7 组装提示词（最后核对）

代码块

```
1 请组装已经分别通过测试的四个文件：requirements.txt、pyproject.toml、
  tool_governance_demo.py、tests/test_tool_governance.py。
2
3 不要重写已经通过测试的业务决策。请完成以下工作：
4 1. 合并对象、PermissionEngine、ToolRuntime、AuditSink 与离线场景；
```

- 5 2. 添加可选 DeepSeek Adapter, Provider 只做协议转换;
- 6 3. 确认 CLI、离线 Demo、pytest、Provider 全部调用同一个 `ToolRuntime.invoke()`;
- 7 4. 输出完整文件, 不使用省略号;
- 8 5. 给出 py_compile、离线运行和 pytest 命令;
- 9 6. 若接口冲突, 先列出冲突, 不得擅自放宽 deny、审批、脱敏或副作用断言。
- 10
- 11 组装后逐项对照八项测试, 不得通过删除测试、修改错误码或把 handler 变成空函数制造通过。

十、本节总结

技术主链

代码块

- 1 不可信 Tool Call
- 2 → Pydantic 与业务预检查
- 3 → deny-first、plan、双重白名单、RBAC 与参数绑定审批
- 4 → 按副作用、幂等和资源关系执行
- 5 → 投影、脱敏、审计和稳定错误码
- 6 → 可验证的 Tool Result

学习主链

代码块

- 1 人定义业务硬性要求
- 2 → 最小规格记录边界
- 3 → Harness 固化唯一执行路径
- 4 → TDD 证明副作用与结果边界
- 5 → Loop 每次增加一条规则
- 6 → AI 加速实现、修复和重构

开发者与 AI 的职责分工:

哪些必须由Agent工程师自己定义? 瑞幸API的幂等性语义, 哪个接口支持幂等哪个不支持。超时阈值, 下单三秒查店一秒。业务状态机, 已取餐不可退款, 已支付不可重复下单。

哪些可以交给AI生成？Pydantic的Schema定义，基于策略表的 `PermissionEngine` 分支样板，审计日志的序列化和脱敏工具类，单元测试的参数化夹具。

验收标准用防守型TDD来卡。测试一，绕过白名单直接传 `cancelOrder`，断言返回拒绝且HTTP调用次数为零。测试二，`createOrder` 模拟超时，断言只调了一次瑞幸API不自动重试。测试三，审批摘要绑定参数，修改金额后重入，断言被拒绝。

治理定义必须紧跟业务语义的变化。同一个接口，昨天是读今天是写，策略就必须跟着变。

| 开发者必须负责 | AI 可以辅助 |
|--|---|
| 定义工具是否值得开放以及真实副作用 | 根据既定接口生成 <code>ToolDefinition</code> 样板 |
| 决定身份、租户、权限和审批的可信来源 | 生成 <code>Pydantic Model</code> 与序列化代码 |
| 规定 <code>deny</code> 、 <code>plan</code> 、 <code>RBAC</code> 、审批和 <code>allow</code> 的顺序 | 按决策表补 <code>PermissionEngine</code> 分支 |
| 判断超时以后真实业务可能处于什么状态 | 实现退避、超时包装和普通错误映射 |
| 设计模型、用户和审计的最小结果视图 | 生成脱敏函数与字段投影样板 |
| 定义副作用证据和回归测试 | 补 <code>pytest</code> 夹具并根据失败结果修复 |
| 对生产行为、业务损失和合规负责 | 提高实现、重构和排障速度 |

十一、课堂练习

10.1 代码生成：补齐 Runtime 的关键检查点

给一个 `invoke()` 的空骨架，能补出：

- 根据工具名查找定义
- `Pydantic` 校验并结构化错误
- `PermissionEngine` 三态决策
- `deny/confirm` 时 `Handler` 零调用
- 根据只读和幂等属性驱动超时与重试
- `finalize` 脱敏与审计
- 返回结果保留原始 `tool_call_id`